

Nama : Yakfi 'Arofah Yuhdika Muftiady

NIM : H1D024114

Shift Awal : C

Shift Baru : B

Studi Kasus 1

1. # 1. Definisi Antecedent (Input) dan Consequent (Output)

Range ditentukan berdasarkan semesta pembicaraan pada soal

```
barang_terjual = ctrl.Antecedent(np.arange(0, 101, 1), 'barang_terjual')
```

```
permintaan = ctrl.Antecedent(np.arange(0, 301, 1), 'permintaan')
```

```
harga_item = ctrl.Antecedent(np.arange(0, 100001, 1), 'harga_item')
```

```
profit = ctrl.Antecedent(np.arange(0, 4000001, 1), 'profit')
```

```
stok_makanan = ctrl.Consequent(np.arange(0, 1001, 1), 'stok_makanan')
```

Di bagian ini, kita mendeklarasikan "apa saja" yang mau kita hitung. Terdapat 4 variabel input (Sebab), yakni **Barang Terjual**, **Permintaan**, **Harga**, dan **Profit**. Semesta pembicaraannya ditentukan di sini (batas nilai minimal dan maksimalnya), seperti `np.arange(0, 101, 1)` untuk barang yang berarti rentang angkanya 0 sampai 100. Sementara itu, satu variabel Output/Hasilnya ditetapkan, yakni **Stok Makanan**.

2. # 2. Membership Functions (Menggunakan fungsi segitiga/trimf sebagai standar)

Barang Terjual [0 - 100]

```
barang_terjual['rendah'] = fuzz.trimf(barang_terjual.universe, [0, 0, 40])
```

```
barang_terjual['sedang'] = fuzz.trimf(barang_terjual.universe, [30, 50, 70])
```

```
barang_terjual['tinggi'] = fuzz.trimf(barang_terjual.universe, [60, 100, 100])
```

Tahap ini mengubah atau "mentransformasikan" bentuk kurva garis di gambar PDF Anda menjadi angka matematika dalam Python. Kita mendefinisikan *membership function*-nya.

- `fuzz.trimf` mengubah variabel menjadi kurva berbentuk **Segitiga** (memiliki titik kiri, puncak kelingking, dan kanan). Misal `[30, 50, 70]` berarti dari 30 ia akan naik menanjak menuju 50, lalu turun menuju 70.
- `fuzz.trapmf` digunakan untuk membentuk **Trapezium** karena memiliki landasan atas yang melandai rata.

3. # 3. Definisi Aturan Fuzzy (Rules)

```
rule1 = ctrl.Rule(barang_terjual['tinggi'] & permintaan['tinggi'] & harga_item['murah'] & profit['banyak'], stok_makanan['banyak'])
```

```

rule2 = ctrl.Rule(barang_terjual['tinggi'] & permintaan['tinggi'] & harga_item['murah'] &
profit['sedang'], stok_makanan['sedang'])
rule3 = ctrl.Rule(barang_terjual['tinggi'] & permintaan['sedang'] & harga_item['murah']
& profit['sedang'], stok_makanan['sedang'])
rule4 = ctrl.Rule(barang_terjual['sedang'] & permintaan['tinggi'] & harga_item['murah']
& profit['sedang'], stok_makanan['sedang'])
rule5 = ctrl.Rule(barang_terjual['sedang'] & permintaan['tinggi'] & harga_item['murah']
& profit['banyak'], stok_makanan['banyak'])
rule6 = ctrl.Rule(barang_terjual['rendah'] & permintaan['rendah'] & harga_item['sedang']
& profit['sedang'], stok_makanan['sedang'])

```

Kita menerjemahkan kalimat kondisi dari bahasa manusia menjadi logika komputer. Ada 6 aturan (Rules) yang tertulis persis di PDF modul. Di sini operator AND ditulis dengan simbol & untuk menggabungkan syarat jika terpenuhi.

4. # 4. Sistem Kontrol dan Simulasi

```

stok_ctrl = ctrl.ControlSystem([rule1, rule2, rule3, rule4, rule5, rule6])
stok_simulation = ctrl.ControlSystemSimulation(stok_ctrl)

```

Sebuah wadah kosong mesin yang dinamakan ControlSystem dibuat untuk *mengumpulkan* sekaligus menampung seluruh kombinasi 6 aturan [rule1, rule2, ..., rule6] yang barusan kita deklarasikan di atas ke dalam satu sistem utuh yang siap digunakan (prediksi_stok).

5. # 5. Memasukkan Nilai Input Sesuai Soal

```

# barang terjual = 80, permintaan = 255, harga per item = Rp 25.000, dan profit = Rp
3.500.000
stok_simulation.input['barang_terjual'] = 80
stok_simulation.input['permintaan'] = 255
stok_simulation.input['harga_item'] = 25000
stok_simulation.input['profit'] = 3500000

```

Di bagian inilah tes "KASUS" sebenarnya baru dimasukkan. Kita menyuplai sistem dengan angka-angka keadaan toko saat itu: Barang berjumlah 80, Permintaan sedang tinggi yakni 255, dan seterusnya. Begitu perintah dewa yaitu **.compute()** terpanggil, sistem Skfuzzy akan menggabungkan seluruh titik irisan kurva, melakukan defuzzifikasi dengan metode *Centroid* (titik berat luasan wilayah himpunan arsir hasil akhir), dan melahirkan angka pasti (Crips) yakni **834.62** unit agar bisa dibaca manusia.

6. # 6. Melakukan Perhitungan (Crushing/Compute)

```
stok_simulation.compute()
```

Terakhir, barisan perintah statemen `.view(...)` bertugas memanggil perangkat pustaka eksternal yakni Matplotlib untuk menggambar plot dari variabel satu per satu secara grafis menyerupai gambar di PDF modul sehingga Anda bisa mempresentasikan bentuk titik dan arsiran aslinya dengan sangat menarik!

Output:

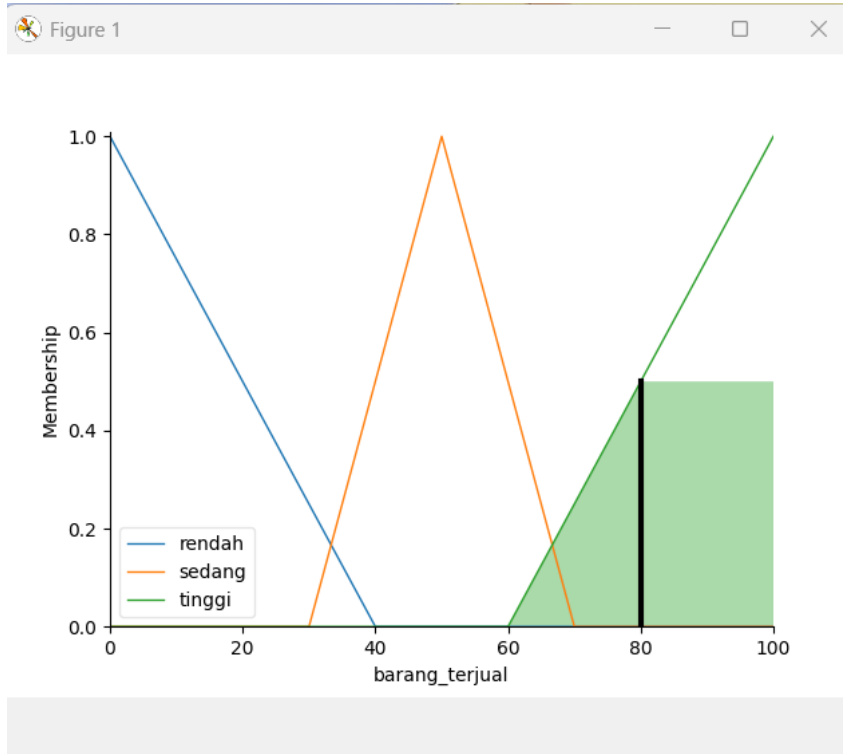


Figure 2

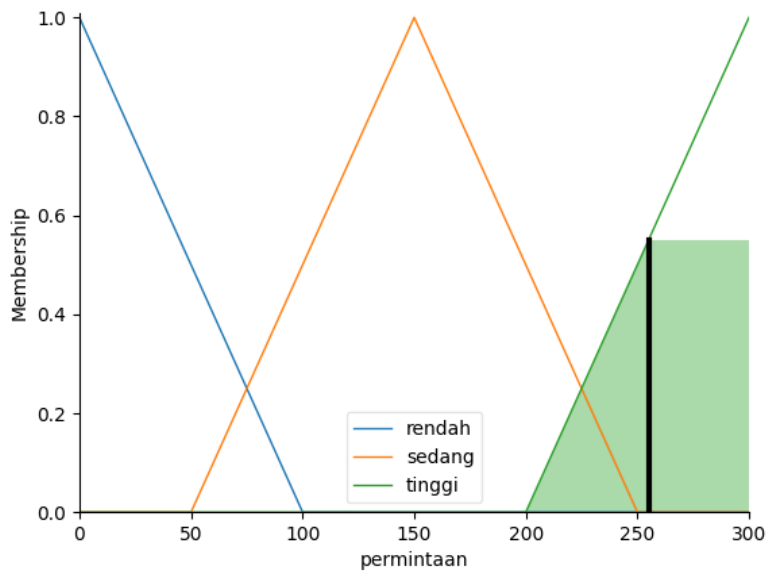


Figure 3

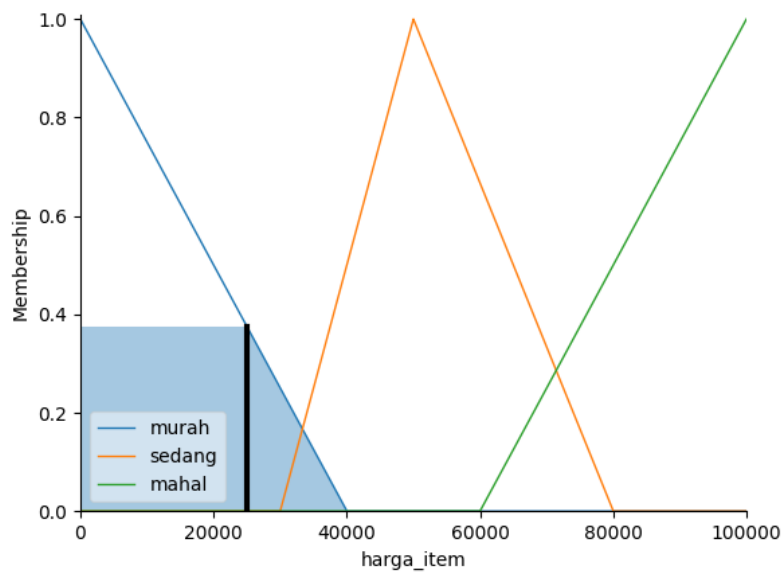


Figure 4

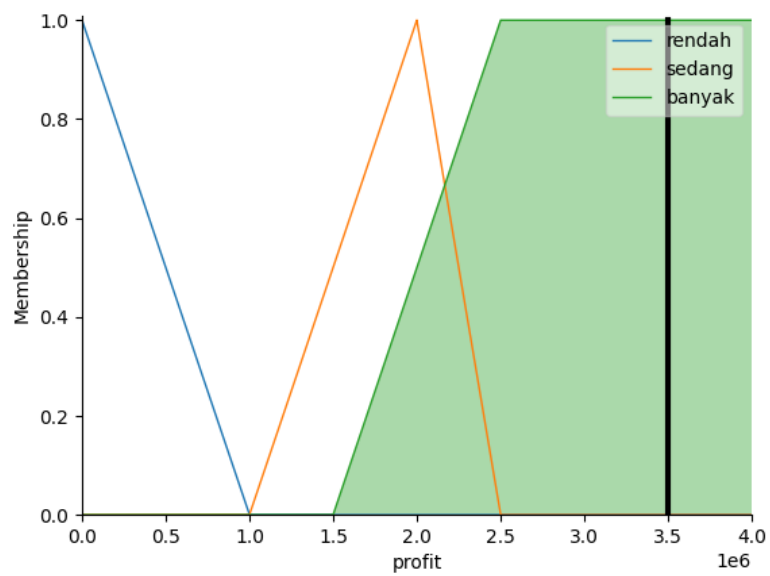
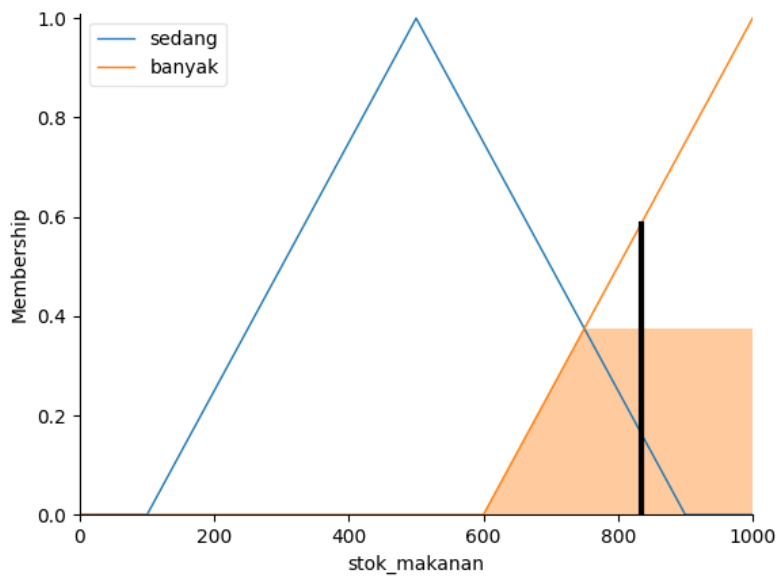


Figure 5 (Not Responding)



Studi Kasus 2:

1. # VARIABEL INPUT (Antecedent)

```
kejelasan_informasi = ctrl.Antecedent(np.arange(0, 101), 'kejelasan_informasi')
kejelasan_persyaratan = ctrl.Antecedent(np.arange(0, 101), 'kejelasan_persyaratan')
kemampuan_petugas = ctrl.Antecedent(np.arange(0, 101), 'kemampuan_petugas')
ketersediaan_sarpras = ctrl.Antecedent(np.arange(0, 101), 'ketersediaan_sarpras')
# VARIABEL MENAMPILKAN HASIL AKHIR (Consequent)
kepuasan_pelayanan = ctrl.Consequent(np.arange(0, 401), 'kepuasan_pelayanan')
```

Di sini kita mendefinisikan apa saja variabel yang akan kita hitung. **Antecedent** adalah variabel "Sebab" (4 kriteria pelayanan yang dinilai dari 0 - 100), dan **Consequent** adalah variabel "Akibat/Output" (Kepuasan Pelayanan yang skalanya 0 - 400).

2. # PEMBENTUKAN HIMPUNAN FUZZY INPUT

KEJELASAN INFORMASI

```
kejelasan_informasi['tidak memuaskan'] = fuzz.trimf(kejelasan_informasi.universe, [0,
60, 75])
kejelasan_informasi['cukup memuaskan'] = fuzz.trimf(kejelasan_informasi.universe, [60,
75, 90])
kejelasan_informasi['memuaskan'] = fuzz.trimf(kejelasan_informasi.universe, [75, 90,
100])
```

Bagian ini menerjemahkan batas titik grafik (sesuai gambar di modul/soal) ke dalam bahasa mesin.

- fuzz.trimf digunakan untuk membentuk kurva himpunan *Triangular* (Segitiga) dengan 3 titik (Mulai, Puncak, Berhenti).
- fuzz.trapmf digunakan untuk membentuk kurva *Trapezoidal* (Trapezium) dengan 4 titik yang memiliki puncak datar.

3. # DAFTAR ATURAN FUZZY (RULES)

```
aturan_1 = ctrl.Rule(kejelasan_informasi['tidak memuaskan'] &
kejelasan_persyaratan['tidak memuaskan'] & kemampuan_petugas['tidak memuaskan'] &
ketersediaan_sarpras['tidak memuaskan'], kepuasan_pelayanan['kurang memuaskan'])
aturan_2 = ctrl.Rule(kejelasan_informasi['tidak memuaskan'] &
kejelasan_persyaratan['tidak memuaskan'] & kemampuan_petugas['tidak memuaskan'] &
ketersediaan_sarpras['cukup memuaskan'], kepuasan_pelayanan['cukup memuaskan'])
aturan_3 = ctrl.Rule(kejelasan_informasi['tidak memuaskan'] &
kejelasan_persyaratan['tidak memuaskan'] & kemampuan_petugas['tidak memuaskan'] &
ketersediaan_sarpras['memuaskan'], kepuasan_pelayanan['cukup memuaskan'])
```

```

aturan_4 = ctrl.Rule(kejelasan_informasi['tidak memuaskan'] &
kejelasan_persyaratan['tidak memuaskan'] & kemampuan_petugas['cukup memuaskan']
& ketersediaan_sarpras['tidak memuaskan'], kepuasan_pelayanan['cukup memuaskan'])
aturan_5 = ctrl.Rule(kejelasan_informasi['tidak memuaskan'] &
kejelasan_persyaratan['tidak memuaskan'] & kemampuan_petugas['cukup memuaskan']
& ketersediaan_sarpras['cukup memuaskan'], kepuasan_pelayanan['cukup memuaskan'])
aturan_6 = ctrl.Rule(kejelasan_informasi['tidak memuaskan'] &
kejelasan_persyaratan['tidak memuaskan'] & kemampuan_petugas['cukup memuaskan']
& ketersediaan_sarpras['memuaskan'], kepuasan_pelayanan['cukup memuaskan'])
aturan_7 = ctrl.Rule(kejelasan_informasi['tidak memuaskan'] &
kejelasan_persyaratan['tidak memuaskan'] & kemampuan_petugas['memuaskan'] &
ketersediaan_sarpras['tidak memuaskan'], kepuasan_pelayanan['cukup memuaskan'])
aturan_8 = ctrl.Rule(kejelasan_informasi['tidak memuaskan'] &
kejelasan_persyaratan['tidak memuaskan'] & kemampuan_petugas['memuaskan'] &
ketersediaan_sarpras['cukup memuaskan'], kepuasan_pelayanan['cukup memuaskan'])
....
aturan_81 = ctrl.Rule(kejelasan_informasi['memuaskan'] &
kejelasan_persyaratan['memuaskan'] & kemampuan_petugas['memuaskan'] &
ketersediaan_sarpras['memuaskan'], kepuasan_pelayanan['sangat memuaskan'])

```

Ini adalah inti dari penalaran (otak) programnya. Terdapat total **81 aturan** (Kombinasi probabilitas dari 3 sifat Info $\times$ 3 Persyaratan $\times$ 3 Petugas $\times$ 3 Sarpras = 81). Kode mengatur prasyarat kondisi "JIKA... MAKA... (IF-THEN)" menggunakan operator AND (&).

4. # PEMBUATAN SISTEM INFERENSI

```

pengatur_fuzzy = ctrl.ControlSystem([aturan_1, aturan_2, aturan_3, aturan_4, aturan_5,
aturan_6, aturan_7, aturan_8, aturan_9, aturan_10, aturan_11, aturan_12, aturan_13,
aturan_14, aturan_15, aturan_16,
aturan_17, aturan_18, aturan_19, aturan_20, aturan_21, aturan_22, aturan_23, aturan_24,
aturan_25, aturan_26, aturan_27, aturan_28, aturan_29, aturan_30, aturan_31,
aturan_32, aturan_33, aturan_34, aturan_35, aturan_36, aturan_37, aturan_38, aturan_39,
aturan_40, aturan_41, aturan_42, aturan_43, aturan_44, aturan_45, aturan_46, aturan_47,
aturan_48, aturan_49, aturan_50,
aturan_51, aturan_52, aturan_53, aturan_54, aturan_55, aturan_56, aturan_57, aturan_58,
aturan_59, aturan_60, aturan_61, aturan_62, aturan_63, aturan_64, aturan_65, aturan_66,
aturan_67, aturan_68, aturan_69,
aturan_70, aturan_71, aturan_72, aturan_73, aturan_74, aturan_75, aturan_76, aturan_77,
aturan_78, aturan_79, aturan_80, aturan_81])

```

```
simulasi_fuzzy = ctrl.ControlSystemSimulation(pengatur_fuzzy)
```

Semua 81 Aturan (Rule 1 sampai 81) yang sudah dijabarkan tadi digabungkan dan dibungkus di dalam sebuah "mesin" bernama `ctrl.ControlSystem`. Setelah itu disimulasikan menggunakan `ControlSystemSimulation` yang bersiap menampung inputan angka.

5. # MEMASUKKAN NILAI INPUT

```
simulasi_fuzzy.input['kejelasan_informasi'] = 80  
simulasi_fuzzy.input['kejelasan_persyaratan'] = 60  
simulasi_fuzzy.input['kemampuan_petugas'] = 50  
simulasi_fuzzy.input['ketersediaan_sarpras'] = 90
```

Di tahap ini, barulah kita menyuntikkan data nilai dari soal kasus ujian kita ke dalam "mesin" simulasi tersebut (Misalnya 80 untuk kejelasan informasi, 60 untuk persyaratan, dsb).

6. # PROSES DEFUZZIFIKASI

```
simulasi_fuzzy.compute()
```

Perintah `.compute()` berfungsi sangat vital. Dia menjalankan kalkulasi rumit untuk mengubah hasil gabungan grafik dari 81 aturan tadi kembali menjadi **1 buah Angka Tegap (Crisp Value)**. Secara standar, Skfuzzy menggunakan metode Centroid (titik berat luasan grafik) untuk menentukan angka ini.

7. # Plotting Grafik

```
kejelasan_informasi.view()  
kejelasan_persyaratan.view()  
kemampuan_petugas.view()  
ketersediaan_sarpras.view()  
kepuasan_pelayanan.view()
```

```
kepuasan_pelayanan.view(sim=simulasi_fuzzy)
```

Fungsi `.view()` dipanggil untuk meminta *matplotlib* "menggambar" visualisasi dari batas kurva dan area arsiran yang sudah dihitung pada mesin defuzzifikasi tadi.

Output:

Figure 1

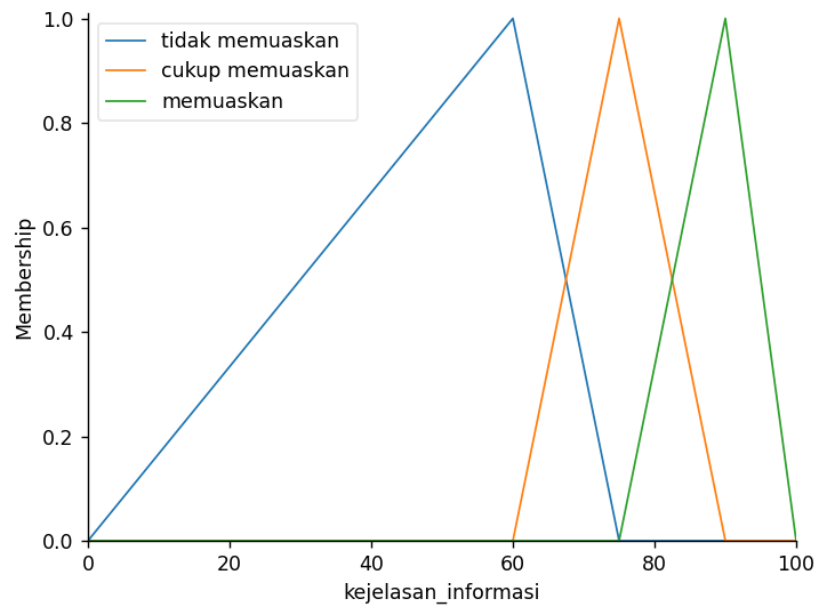
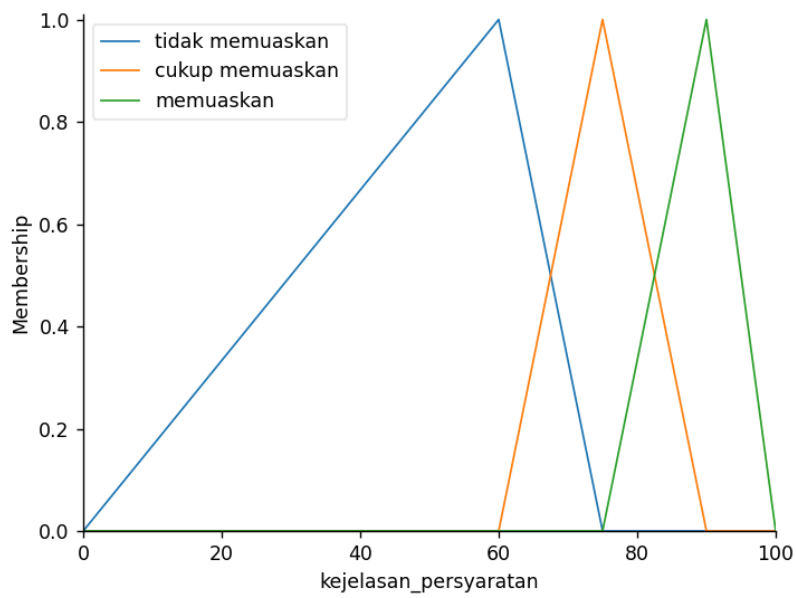


Figure 2



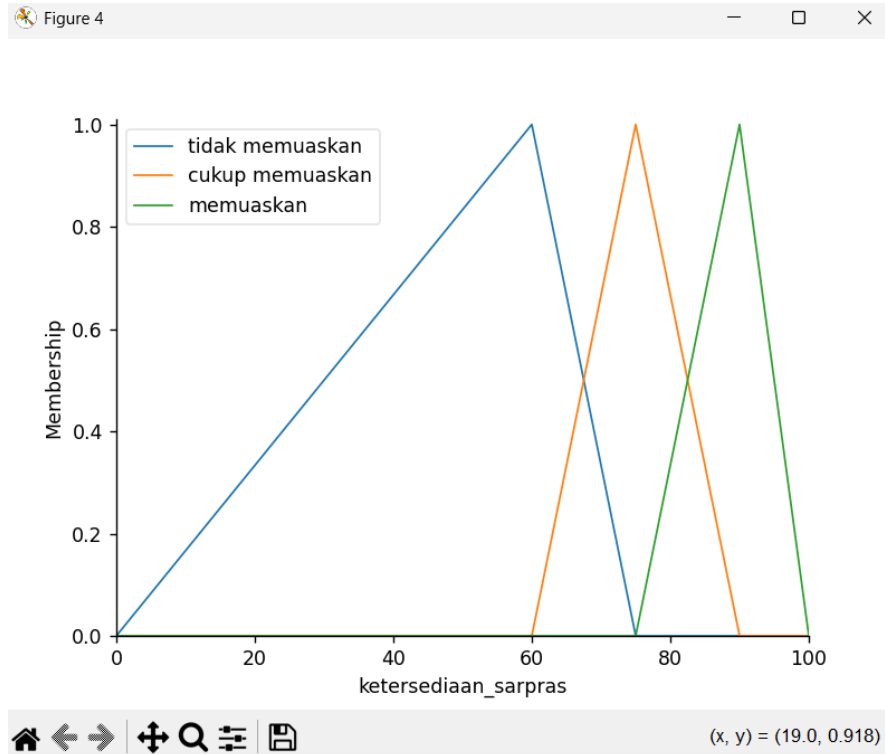
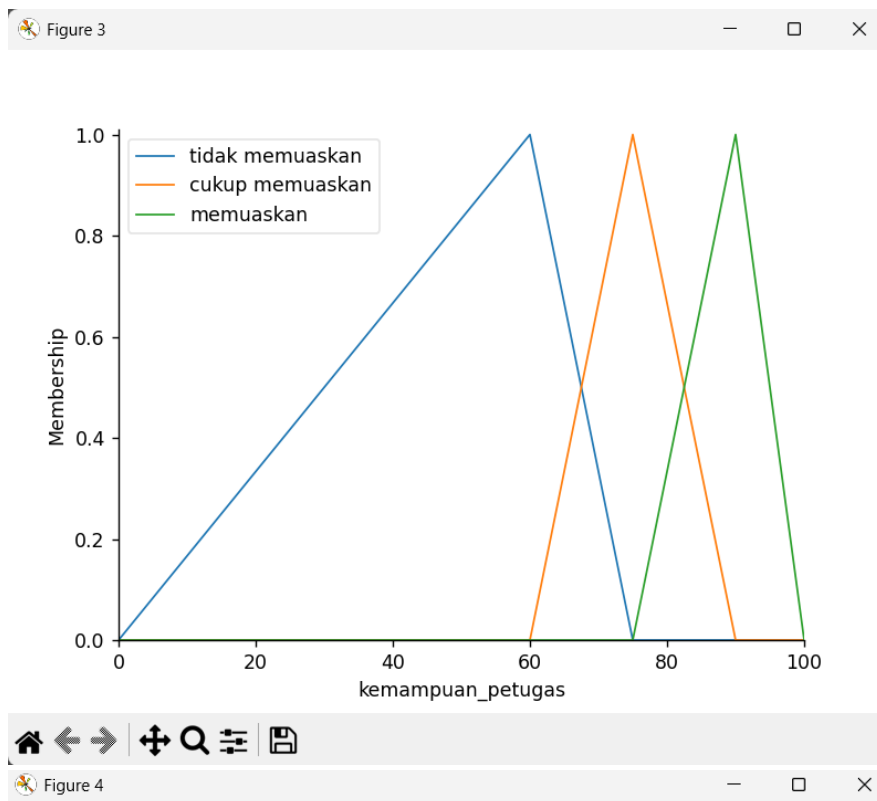


Figure 5

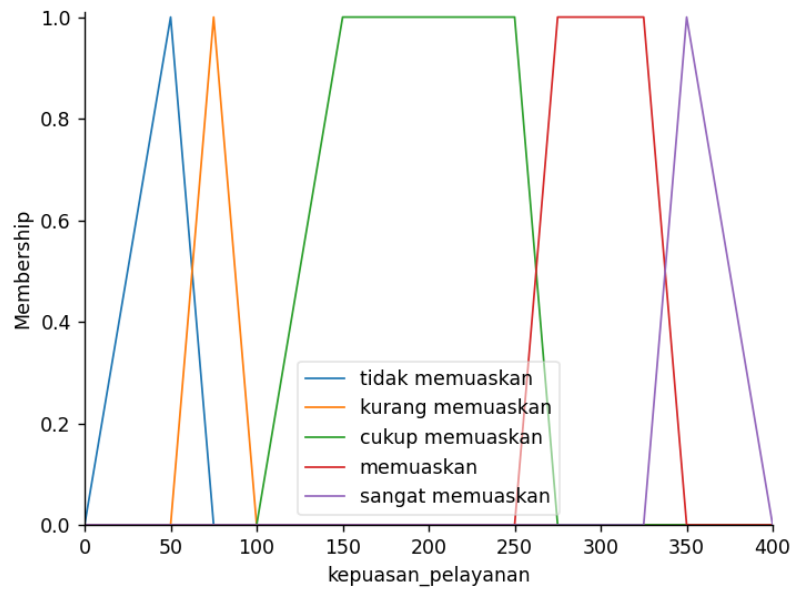


Figure 6

